Improving interactivity and consistency of the Rzk Language Server through on-edit reevaluation.

Student: Vyacheslav Molchanov
Supervisor: Nikolai Kudasov

Abstract

Modern software development relies heavily on Integrated Development Environments (IDEs) and code editors powered by the Language Server Protocol (LSP). While the LSP standardizes the separation between development tools and language itself, existing implementations often face scalability and feature bottlenecks when handling massive codebases, leading to frustrating user experience. This paper presents the design, implementation, and evaluation of an enhanced LSP server for Rzk proof assistant. It introduces architectural advancements to type checking and caching: allowing typechecker to read buffer of unsaved user edits on codebase and incremental building. Mentioned advancements makes IDE more responsive and reduces latency on big codebases.

1 Introduction

In recent years, the ecosystem of software development tools has shifted toward a decoupled architecture. Historically, adding programming language support to an editor required building compiler-like static analyzers unique to that specific interface. This led to a combinatorial explosion of development effort across different IDEs. The introduction of the Language Server Protocol (LSP) by Microsoft [1] and industry partners revolutionized this domain by formalizing a standard, JSON-RPC-based communication mechanism between development clients and language-specific servers. Consequently, features such as autocompletion, code navigation, and real-time diagnostics can now be implemented once per language and instantly deployed across multiple modern editors.

Rzk is a proof assistant based on Riehl and Shulman's type theory [3] with shapes. It has LSP server implementation, however typechecker of Rzk LSP server is not aware of unsaved IDE changes and rebuilds all code after changed region, leading to latency on big projects and unresponsive user experience. To address it, this study aims to design and implement incremental build system [2] for Rzk LSP server.

2 Novelty and Contributions

This project provides Rzk LSP server with several features:

- **On-edit diagnostics:** The typechecker module of language server will compile the rzk codebase with respect to unsaved changes from IDE. Typechecking will be rerunned on every user input, improving IDE responsiveness.
- **Incremental building:** The typechecker will rebuild only the code, that changed since previous run and its dependencies, lowering diagnostic latency.

3 State-of-the-Art

There are numerous examples of proof assistants with LSP (sometimes it features additional functionality to LSP), for example:

- **Agda:** Agda is a dependently typed programming language and it is an extension of Martin-Löf's type theory. Despite the fact that agda does not feature full incremental typechecking, it caches the checked files and do not recheck them if not changed.
- **Lean4:** Lean is a functional programming language and theorem prover built for formalizing math and for formal verification. Lean have similar caching on scale of multiple files, however for a single file it caches every declaration and invalidates only the rest of the file below the changed code. Also Lean have optimization if changes only touched the body of the declaration: Lean does not recheck whole declaration if signature is unchanged and only check if body matches the signature.
- **Rocq:** Roqc prover is another interactive theorem prover. Its lsp (known as coq-lsp) has declaration dependency graph which allows fast incremental builds.

4 Goal of the Thesis

The main goal is to design, implement and integrate incremental build system for Rzk LSP server that will provide relevant diagnostics with low latencies. As a result, the LSP server should:

- Update diagnostics with respect to user changes, providing responsiveness.
- Rechecking only changed declarations or declarations, dependent on them, lowering latencies.

Achieving these goals will lead to improving Rzk user experience.

5 Detailed Work Plan and Schedule

The research will be done in two semesters. The first semester will be dedicated to developing build system. It also will include making typechecker aware of unsaved changes and triggerring rebuild on user input. The second semester will focus on testing and benchmarking the solution, making sure that build system is minimal and correct. All these steps will be documented in parralel.

Semester 1:

- **Explore codebase and design improvements(September - November)**

  The codebase should be explored carefully to understand overall project architecture. Future improvements should be designed with respect with current codebase.

- **Implement the improvements(November - December)**
  - Make typecheker aware of unsaved changes.
  - Implement cache invalidation.

Semester 2:

- **Testing and benchmarking (January-March):**
  - Verifying the correctness of builds.
  - Measuring latencies with/without incremental build system.
- **Real-world tests (March-April):**

  Distribute updated Rzk LSP Server inside Rzk community, collect and analyze feedback.

- **Summarization of the work and preparation for the demo (May):**
  - Finalize and polish the thesis work.
  - Prepare for the defense.
  - Submit thesis to a relevant conference or journal.

6 Conclusion

The development of build system for Rzk LSP server aims to remove repetative type-checking of unchanged code. The expected outcome is LSP server that responds fast with correct code diagnostics. Overall, the project purpose is to improve Rzk user expirience.

References

[1] Microsoft Corporation. Language server protocol specification, 2026.

[2] Andrey Mokhov, Neil Mitchell, and Simon Peyton Jones. Build systems à la carte. *Proc. ACM Program. Lang.*, 2(ICFP), July 2018.

[3] Emily Riehl and Michael Shulman. A type theory for synthetic ∞-categories. 2023.